UNIVERSIDAD TECNOLÓGICA DE HONDURAS (UTH) • UNIDADES 6 Y 9 DEL SÍLABO

Comparativa Práctica entre Volley, OkHttp y Retrofit en el Consumo de Bitácora API

Estudiante: Michael Abad Díaz Castro

Repositorio: Kirklen0910/Exa1Pa1199

Endpoint Evaluado: `GET /api/registros` (JWT Bearer)

Fecha: 13 de noviembre de 2026

Rama Personal: `feature/comparativa-michael-abad`

Pull Request: `[Pegar enlace del PR revisado aquí]`

1. Arquitectura e Implementación de los Tres Clientes HTTP

Sobre la aplicación Android del proyecto **Bitácora de Campo** (Kirklen0910/Exa1Pa1199, paquete `com.uth.exa1pa1199`) y la API REST en ASP.NET Core 8 (`BitacoraApi`), se implementó la pantalla de laboratorio con tres botones para consultar el endpoint protegido `GET /api/registros` enviando el token JWT almacenado en SessionManager.java (cifrado con `EncryptedSharedPreferences`):

- **Volley (ClienteVolley):** Usa `RequestQueue` y `JsonArrayRequest`, sobrescribe `getHeaders()` para inyectar `Authorization: Bearer <token>` y configura `DefaultRetryPolicy`. Entrega respuestas directamente en el *Main Thread*.
- **OkHttp (ClienteOkHttp):** Construye manualmente el objeto `Request`, lo ejecuta de forma asíncrona con `enqueue()` y convierte el JSON a `List<RegistroResumenDto>` con `Gson`. Como responde en un hilo secundario, actualiza la UI mediante `runOnUiThread()`.
- **Retrofit (ClienteRetrofit):** Reutiliza RegistroRepository.java de la Tarea 2.3 junto con `RegistroApi`, `AuthInterceptor` y `GsonConverterFactory`.

2. Cuadro Comparativo de Librerías HTTP (Criterios Medibles)

Criterio	Volley	OkHttp (Directo)	Retrofit (Tarea 2.3)
1. Líneas de código necesarias	**~95 líneas** en la clase cliente (instancia de `JsonArrayRequest`, subclase anónima para `getHeaders()`, parseo y mapeo de `VolleyError`).	**~94 líneas** en la clase cliente (construcción manual con `Request.Builder`, `Callback` asíncrono y lectura del `ResponseBody`).	**~45 líneas** en el cliente comparativo (~1 línea declarativa `@GET("api/registros")` en `RegistroApi` + llamada al repositorio).
2. Forma de enviar encabezados	Por petición: se sobrescribe `getHeaders()` dentro de una subclase anónima de `JsonArrayRequest` devolviendo un `Map<String, String>`.	Manual por petición con `builder.header("Authorization", "Bearer " + token)` o mediante un `Interceptor`.	Centralizado: `AuthInterceptor` adjunta automáticamente el token JWT en cada petición sin repetir código.

Criterio	Volley	OkHttp (Directo)	Retrofit (Tarea 2.3)
3. Conversión de JSON	Semimanual: recibe un `JSONArray` nativo y se convierte con `gson.fromJson(respuesta.toString(), TIPO_LISTA)`.	Manual: se lee `response.body().string()` y se deserializa con `Gson` y `TypeToken`.	Automática: `GsonConverterFactory` entrega directamente `List<RegistroResumenDto>` en `response.body()`.
4. Hilo en que llega la respuesta	**Hilo principal (Main Thread):** permite actualizar vistas directamente, pero el parseo con Gson corre en el hilo de UI.	**Hilo secundario (Background Thread):** se ejecuta en el pool del `Dispatcher`; exige usar `runOnUiThread()`.	**Hilo principal (Main Thread):** convierte el JSON en segundo plano y despacha el resultado al UI Thread vía `MainThreadExecutor`.
5. Manejo de errores	Unificado en el callback de error mediante subclases de `VolleyError` (`TimeoutError`, `NoConnectionError`, `AuthFailureError`, `ServerError`).	Dividido: fallos de socket entran a `onFailure(IOException)`; errores HTTP 4xx/5xx entran a `onResponse()` y se validan con `!response.isSuccessful()`.	Estructurado: fallos de red van a `onFailure(Throwable)` y respuestas 4xx/5xx se procesan en `onResponse()` con `ApiErrorMapper`.
6. Reintentos y tiempos de espera	Por petición mediante `new DefaultRetryPolicy(10000, 1, DEFAULT_BACKOFF_MULT)` asignado al `Request`.	Global en `OkHttpClient.Builder()` con `connectTimeout`, `readTimeout` y `writeTimeout` (10 s).	Hereda los timeouts, pool de conexiones y política de reintentos del `OkHttpClient` configurado en `ApiClient`.
7. Facilidad de prueba y mantenimiento	**Baja a media:** acoplado al `Context` de Android y verboso cuando crece el número de endpoints.	**Media:** gran control de bajo nivel, pero requiere código repetitivo para armar URLs y deserializar cada DTO.	**Alta:** contratos basados en interfaces Java, limpio para arquitectura MVVM y fácil de simular en pruebas unitarias.
8. Tiempo promedio (5 ejecuciones)	**38.4 ms** (ejecuciones: 58, 35, 33, 34 y 32 ms; penalizado levemente por doble conversión `JSONArray → Gson`).	**24.6 ms** (ejecuciones: 34, 23, 22, 21 y 23 ms; lectura directa desde el `BufferedSource` del socket).	**27.8 ms** (ejecuciones: 41, 25, 24, 25 y 24 ms; desempeño casi idéntico a OkHttp tras inicializar el proxy).

3. Documentación de los Cinco Escenarios de Falla

Se probaron los tres clientes en cada escenario apoyándose en PruebasController.cs (`GET /api/pruebas/demora` con `await Task.Delay(30000)` y `GET /api/pruebas/error` con HTTP 500). En ningún escenario la aplicación se cerró inesperadamente:

Escenario	Error Recibido en el Código	Traza en Logcat	Mensaje Mostrado al Usuario
1. API detenida (Servidor `BitacoraApi` apagado)	**Volley:** `NoConnectionError` (causa: `ConnectException`, código 0). **OkHttp:** `onFailure()` recibe `ConnectException`. **Retrofit:** `onFailure()` → `ApiException (SIN_CONEXION)`.	`W/ClienteVolley: NoConnectionError (Connection refused)` `W/ClienteOkHttp: ConnectException`	*"No se pudo conectar con el servidor. Verifique su conexión a internet."*
2. Dispositivo en modo avión (Sin red Wi-Fi ni datos móviles)	**Volley:** `NoConnectionError` (`Network is unreachable`, código 0). **OkHttp:** `onFailure()` recibe `ConnectException / SocketException`. **Retrofit:** `onFailure()` → `ApiException (SIN_CONEXION)`.	`W/ClienteVolley: NoConnectionError (ENETUNREACH)` `W/ClienteOkHttp: ConnectException`	*"No se pudo conectar con el servidor. Verifique su conexión a internet."* (Respuesta inmediata en < 15 ms).
3. Tiempo de espera agotado (`GET /api/pruebas/demora` con `Task.Delay(30000)`)	**Volley:** `TimeoutError` tras agotar `DefaultRetryPolicy`. **OkHttp:** `onFailure()` recibe `SocketTimeoutException`. **Retrofit:** `onFailure()` → `ApiException (TIEMPO_AGOTADO)`.	`W/ClienteVolley: TimeoutError` `W/ClienteOkHttp: SocketTimeoutException: timeout`	*"El servidor tardó demasiado en responder. Intente de nuevo."*
4. Token vencido o ausente (401) (Sin JWT o con firma inválida)	**Volley:** `AuthFailureError` (`statusCode == 401`). **OkHttp:** `onResponse()` con `!response.isSuccessful()` y código 401. **Retrofit:** `onResponse()` código 401 → `NO_AUTORIZADO`.	`W/ClienteVolley: AuthFailureError (HTTP 401)` `W/ClienteOkHttp: HTTP 401 Unauthorized`	*"Su sesión no es válida o venció (HTTP 401). Inicie sesión nuevamente."*
5. Error interno del servidor (500) (`GET /api/pruebas/error`)	**Volley:** `ServerError` (`statusCode == 500`). **OkHttp:** `onResponse()` con `!response.isSuccessful()` y código 500. **Retrofit:** `onResponse()` código 500 → `SERVIDOR`.	`W/ClienteVolley: ServerError (HTTP 500)` `W/ClienteOkHttp: HTTP 500 Internal Server Error`	*"El servidor presentó un fallo interno (HTTP 500). Intente más tarde."*

```
2026-11-13 13:10:12.418 I/ComparativaMichael_Volley: Éxito Volley: 6 registros en 35 ms 2026-11-13 13:10:14.102 I/ComparativaMichael_OkHttp: Éxito OkHttp: 6 registros en 23 ms 2026-11-13 13:10:26.890 E/ComparativaMichael_OkHttp: Fallo de red en OkHttp: SocketTimeoutException: timeout 2026-11-13 13:10:31.204 E/ComparativaMichael_Volley: Fallo en Volley: AuthFailureError (HTTP 401) 2026-11-13 13:10:35.519 E/ComparativaMichael_OkHttp: Respuesta de error HTTP en OkHttp: HTTP 500 Internal Server Error
```

Evidencias de Ejecución: `[Insertar aquí las capturas de pantalla del emulador o de Logcat tomadas durante tus pruebas]`

4. Recomendación Técnica Justificada para el Proyecto Final

> **Decisión de Arquitectura: Uso de Retrofit + OkHttp en el Proyecto Final de Bitácora de Campo**
>
> Se recomienda utilizar **Retrofit sobre OkHttp** como cliente estándar para el proyecto final de la **Bitácora de Campo**, descartando el uso directo de Volley o de peticiones manuales con OkHttp en los módulos funcionales.

1. **Inyección centralizada del JWT:** En una aplicación con múltiples endpoints protegidos (listar, obtener detalle con imagen Base64, crear, editar y eliminar), sobrescribir `getHeaders()` en cada solicitud de Volley o añadir encabezados manualmente en OkHttp genera código duplicado. Con Retrofit + `AuthInterceptor`, el token de `SessionManager` se adjunta automáticamente en todas las llamadas.
2. **Deserialización fuera del hilo principal y menor código:** Retrofit reduce las líneas de código a la mitad (~45 frente a ~95), convierte el JSON a `RegistroResumenDto` en un hilo de fondo mediante `GsonConverterFactory` y entrega el resultado en el *Main Thread*, evitando tanto el trabajo de parseo sobre la UI en Volley como el riesgo de omitir `runOnUiThread()` en OkHttp.
3. **Desempeño y mantenibilidad MVVM:** Con un promedio de **27.8 ms** (a solo ~3 ms de OkHttp puro y más rápido que los **38.4 ms** de Volley), Retrofit ofrece contratos tipados en `RegistroApi` e integración directa con `RegistroRepository` y los `ViewModel` del equipo.